

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An autonomous Institution affiliated to Anna University)

M. Tech (Integrated) Computer Science & Engineering

Semester V

ICS1512 – Machine Learning Algorithms Laboratory

Experiment 5

Decision Tree and Random Forest:

A Comparative Classification Study

NATHANIEL CHRISTIAN

3122247001037

Degree & Branch	M. Tech (Integrated) Computer Science & Engineering
Semester	V
Subject Code	ICS1512
Academic Year	2026–2027 (Odd)
Batch	2024–2029
Due Date	_____

Contents

1	Experiment 5	3
1.1	Title	3
1.2	Objective	3
2	Dataset	3
2.1	Wisconsin Diagnostic Breast Cancer Dataset	3
3	Theory	3
3.1	Decision Tree Classifier	3
3.1.1	Gini Index	4
3.1.2	Entropy	4
3.2	Random Forest Classifier	4
4	Steps for Implementation	4
5	Hyperparameters Explored	5
5.1	Decision Tree	5
5.2	Random Forest	5
6	Python Implementation	5
6.1	Import Libraries	5
6.2	Load and Prepare Dataset	6
6.3	Exploratory Data Analysis	6
6.4	Train-Test Split	7
7	Decision Tree	7
7.1	Initial Model	7
7.2	5-Fold Hyperparameter Selection	7
8	Random Forest	8
8.1	Initial Model	8
8.2	5-Fold Hyperparameter Selection	8
9	5-Fold Cross-Validation	9
10	Hyperparameter Tuning Results	10
10.1	Table 1: Decision Tree Hyperparameter Evaluation	10
10.2	Table 2: Random Forest Hyperparameter Evaluation	10

11 Evaluation Metrics	10
11.1 Evaluation Code	11
12 Confusion Matrix	12
13 ROC Curve and AUC	12
14 5-Fold Cross-Validation Performance Comparison	13
15 Final Model Comparison	14
16 Observation Questions	14
16.1 How does tree depth affect overfitting in Decision Trees?	14
16.2 Which hyperparameter had the greatest impact on performance?	14
16.3 How does Random Forest improve generalization?	14
16.4 Did ensemble learning always improve performance? Why or why not?	14
17 Result	15
18 Conclusion	15
19 References	15

1 Experiment 5

1.1 Title

Decision Tree and Random Forest: A Comparative Classification Study

1.2 Objective

1. To implement a Decision Tree classifier.
2. To extend the Decision Tree into a Random Forest ensemble model.
3. To study the impact of hyperparameters on overfitting and generalization.
4. To select optimal hyperparameters using 5-Fold Cross-Validation.
5. To compare single-tree and ensemble-tree models.

2 Dataset

2.1 Wisconsin Diagnostic Breast Cancer Dataset

- **Total samples:** 569
- **Features:** 30 numerical attributes
- **Target classes:** Malignant (M) and Benign (B)
- **Source:** UCI Machine Learning Repository
- **Dataset link:** <https://archive.ics.uci.edu>

3 Theory

3.1 Decision Tree Classifier

A Decision Tree is a supervised learning model that recursively splits the feature space using impurity measures to form decision rules.

- Gini Index and Entropy
- Tree depth and node splitting
- Overfitting in deep trees

3.1.1 Gini Index

$$G = 1 - \sum_{i=1}^C p_i^2$$

3.1.2 Entropy

$$H = - \sum_{i=1}^C p_i \log_2(p_i)$$

Decision Trees can have high variance and are prone to overfitting when they become excessively deep.

3.2 Random Forest Classifier

Random Forest is an ensemble learning technique that combines multiple decision trees trained using bootstrapped samples.

- Bootstrap aggregation (bagging)
- Random feature selection
- Majority voting

Random Forest generally reduces variance and improves generalization compared with a single Decision Tree.

4 Steps for Implementation

1. Load the dataset and encode class labels.
2. Perform Exploratory Data Analysis: class distribution and feature correlation analysis.
3. Split the dataset into training and testing sets using an 80–20 split.
4. Train a Decision Tree classifier.
5. Define a hyperparameter search space.
6. Perform 5-Fold Cross-Validation to evaluate each hyperparameter combination.
7. Select the hyperparameters that yield the best average cross-validation performance.
8. Train a Random Forest classifier.
9. Repeat cross-validation-based hyperparameter selection.
10. Compare both models using evaluation metrics.

5 Hyperparameters Explored

5.1 Decision Tree

- `criterion`
- `max_depth`
- `min_samples_split`
- `min_samples_leaf`

5.2 Random Forest

- `n_estimators`
- `max_depth`
- `max_features`
- `bootstrap`

6 Python Implementation

6.1 Import Libraries

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from sklearn.model_selection import (
7     train_test_split, StratifiedKFold,
8     GridSearchCV, cross_validate
9 )
10 from sklearn.preprocessing import LabelEncoder
11 from sklearn.tree import DecisionTreeClassifier
12 from sklearn.ensemble import RandomForestClassifier
13 from sklearn.metrics import (
14     accuracy_score, precision_score, recall_score,
15     f1_score, confusion_matrix, classification_report,
16     roc_curve, roc_auc_score
17 )
```

6.2 Load and Prepare Dataset

```
1 df = pd.read_csv("data.csv")
2
3 print(df.shape)
4 print(df.head())
5 print(df.info())
6
7 # Remove non-feature columns if present
8 if "id" in df.columns:
9     df = df.drop(columns=["id"])
10
11 if "Unnamed: 32" in df.columns:
12     df = df.drop(columns=["Unnamed: 32"])
13
14 # Encode M/B target
15 encoder = LabelEncoder()
16 df["diagnosis"] = encoder.fit_transform(df["diagnosis"])
17
18 X = df.drop(columns=["diagnosis"])
19 y = df["diagnosis"]
20
21 print("Classes:", encoder.classes_)
22 print("Features:", X.shape)
```

6.3 Exploratory Data Analysis

```
1 print(df["diagnosis"].value_counts())
2
3 plt.figure(figsize=(6,4))
4 sns.countplot(x="diagnosis", data=df)
5 plt.title("Class Distribution")
6 plt.xlabel("Class")
7 plt.ylabel("Count")
8 plt.tight_layout()
9 plt.show()
10
11 plt.figure(figsize=(16,12))
12 sns.heatmap(X.corr(), cmap="coolwarm", center=0)
13 plt.title("Feature Correlation Matrix")
14 plt.tight_layout()
```

```
15 plt.show()
```

6.4 Train-Test Split

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y,
3     test_size=0.20,
4     random_state=42,
5     stratify=y
6 )
7
8 print("Training samples:", len(X_train))
9 print("Testing samples:", len(X_test))
```

7 Decision Tree

7.1 Initial Model

```
1 dt_model = DecisionTreeClassifier(
2     criterion="gini",
3     random_state=42
4 )
5 dt_model.fit(X_train, y_train)
6 dt_pred = dt_model.predict(X_test)
7
8 print("Decision Tree Accuracy:",
9     accuracy_score(y_test, dt_pred))
```

7.2 5-Fold Hyperparameter Selection

```
1 dt_param_grid = {
2     "criterion": ["gini", "entropy"],
3     "max_depth": [2, 3, 4, 5, 6, 8, 10, None],
4     "min_samples_split": [2, 5, 10],
5     "min_samples_leaf": [1, 2, 4]
6 }
7
8 cv = StratifiedKFold(
9     n_splits=5,
```

```

10     shuffle=True,
11     random_state=42
12 )
13
14 dt_grid = GridSearchCV(
15     DecisionTreeClassifier(random_state=42),
16     dt_param_grid,
17     scoring="f1",
18     cv=cv,
19     n_jobs=-1,
20     return_train_score=True
21 )
22
23 dt_grid.fit(X_train, y_train)
24
25 print("Best DT parameters:", dt_grid.best_params_)
26 print("Best DT CV F1:", dt_grid.best_score_)
27 best_dt = dt_grid.best_estimator_

```

8 Random Forest

8.1 Initial Model

```

1 rf_model = RandomForestClassifier(
2     n_estimators=100,
3     random_state=42,
4     n_jobs=-1
5 )
6 rf_model.fit(X_train, y_train)
7 rf_pred = rf_model.predict(X_test)
8
9 print("Random Forest Accuracy:",
10     accuracy_score(y_test, rf_pred))

```

8.2 5-Fold Hyperparameter Selection

```

1 rf_param_grid = {
2     "n_estimators": [50, 100, 200],
3     "max_depth": [3, 5, 8, 10, None],
4     "max_features": ["sqrt", "log2"],

```

```

5     "bootstrap": [True, False]
6 }
7
8 rf_grid = GridSearchCV(
9     RandomForestClassifier(
10         random_state=42, n_jobs=-1
11     ),
12     rf_param_grid,
13     scoring="f1",
14     cv=cv,
15     n_jobs=-1,
16     return_train_score=True
17 )
18
19 rf_grid.fit(X_train, y_train)
20
21 print("Best RF parameters:", rf_grid.best_params_)
22 print("Best RF CV F1:", rf_grid.best_score_)
23 best_rf = rf_grid.best_estimator_

```

9 5-Fold Cross-Validation

Five-fold cross-validation divides the training data into five folds. Four folds are used for training and one fold for validation in each iteration. Every fold is used once as the validation fold.

$$\text{Average CV Score} = \frac{1}{5} \sum_{i=1}^5 S_i$$

```

1 dt_cv = cross_validate(
2     best_dt, X_train, y_train,
3     cv=cv,
4     scoring=["accuracy", "precision", "recall", "f1"]
5 )
6
7 rf_cv = cross_validate(
8     best_rf, X_train, y_train,
9     cv=cv,
10    scoring=["accuracy", "precision", "recall", "f1"]
11 )
12

```

```

13 print("Decision Tree Fold Accuracy:",
14       dt_cv["test_accuracy"])
15 print("DT Average:",
16       dt_cv["test_accuracy"].mean())
17
18 print("Random Forest Fold Accuracy:",
19       rf_cv["test_accuracy"])
20 print("RF Average:",
21       rf_cv["test_accuracy"].mean())

```

10 Hyperparameter Tuning Results

10.1 Table 1: Decision Tree Hyperparameter Evaluation

Table 1: Decision Tree Hyperparameter Evaluation using 5-Fold Cross-Validation

Criterion	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
Gini			
Gini			
Entropy			
Entropy			

Fill the table with the actual values obtained from `dt_grid.cv_results_`.

10.2 Table 2: Random Forest Hyperparameter Evaluation

Table 2: Random Forest Hyperparameter Evaluation using 5-Fold Cross-Validation

n Estimators	Max Depth	Max Features	Avg CV Accuracy (%)	Avg CV F1 Score
50				
100				
200				

Fill the table with the actual values obtained from `rf_grid.cv_results_`.

11 Evaluation Metrics

The models are evaluated using Accuracy, Precision, Recall, F1-score, Confusion Matrix, ROC Curve and AUC.

Let TP , TN , FP and FN denote true positive, true negative, false positive and false negative counts.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \left(\frac{Precision \times Recall}{Precision + Recall} \right)$$

11.1 Evaluation Code

```

1 def evaluate_model(name, model, X_test, y_test):
2     pred = model.predict(X_test)
3     prob = model.predict_proba(X_test)[: , 1]
4
5     accuracy = accuracy_score(y_test, pred)
6     precision = precision_score(y_test, pred)
7     recall = recall_score(y_test, pred)
8     f1 = f1_score(y_test, pred)
9     auc = roc_auc_score(y_test, prob)
10
11     print("\n", name)
12     print("Accuracy :", accuracy)
13     print("Precision:", precision)
14     print("Recall   :", recall)
15     print("F1 Score :", f1)
16     print("AUC      :", auc)
17
18     print("\nClassification Report:")
19     print(classification_report(y_test, pred))
20
21     print("\nConfusion Matrix:")
22     print(confusion_matrix(y_test, pred))
23
24     return pred, prob
25
26 dt_pred, dt_prob = evaluate_model(

```

```

27     "Decision Tree", best_dt, X_test, y_test
28 )
29
30 rf_pred, rf_prob = evaluate_model(
31     "Random Forest", best_rf, X_test, y_test
32 )

```

12 Confusion Matrix

```

1 fig, axes = plt.subplots(1, 2, figsize=(11,4))
2
3 sns.heatmap(
4     confusion_matrix(y_test, dt_pred),
5     annot=True, fmt="d", cmap="Blues",
6     ax=axes[0]
7 )
8 axes[0].set_title("Decision Tree")
9 axes[0].set_xlabel("Predicted")
10 axes[0].set_ylabel("Actual")
11
12 sns.heatmap(
13     confusion_matrix(y_test, rf_pred),
14     annot=True, fmt="d", cmap="Blues",
15     ax=axes[1]
16 )
17 axes[1].set_title("Random Forest")
18 axes[1].set_xlabel("Predicted")
19 axes[1].set_ylabel("Actual")
20
21 plt.tight_layout()
22 plt.show()

```

The confusion matrix shows correctly and incorrectly classified samples for both classes and helps identify false positives and false negatives.

13 ROC Curve and AUC

```

1 dt_fpr, dt_tpr, _ = roc_curve(y_test, dt_prob)
2 rf_fpr, rf_tpr, _ = roc_curve(y_test, rf_prob)
3

```

```

4 dt_auc = roc_auc_score(y_test, dt_prob)
5 rf_auc = roc_auc_score(y_test, rf_prob)
6
7 plt.figure(figsize=(7,5))
8 plt.plot(dt_fpr, dt_tpr,
9          label=f"Decision Tree (AUC={dt_auc:.3f})")
10 plt.plot(rf_fpr, rf_tpr,
11          label=f"Random Forest (AUC={rf_auc:.3f})")
12 plt.plot([0,1], [0,1], linestyle="--")
13
14 plt.xlabel("False Positive Rate")
15 plt.ylabel("True Positive Rate")
16 plt.title("ROC Curve Comparison")
17 plt.legend()
18 plt.tight_layout()
19 plt.show()

```

14 5-Fold Cross-Validation Performance Comparison

Table 3: 5-Fold Cross-Validation Accuracy Comparison

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree						
Random Forest						

```

1 print("Decision Tree")
2 for i, score in enumerate(dt_cv["test_accuracy"], 1):
3     print(f"Fold {i}: {score:.4f}")
4 print("Average:", dt_cv["test_accuracy"].mean())
5
6 print("\nRandom Forest")
7 for i, score in enumerate(rf_cv["test_accuracy"], 1):
8     print(f"Fold {i}: {score:.4f}")
9 print("Average:", rf_cv["test_accuracy"].mean())

```

15 Final Model Comparison

Table 4: Final Test-Set Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	AUC
Decision Tree					
Random Forest					

Enter the actual numerical values produced by the executed program.

16 Observation Questions

16.1 How does tree depth affect overfitting in Decision Trees?

Increasing tree depth allows the Decision Tree to learn increasingly complex decision boundaries. Very deep trees can learn noise and training-specific patterns, causing overfitting and poorer generalization. Limiting `max_depth` helps control model complexity.

16.2 Which hyperparameter had the greatest impact on performance?

This should be determined from the actual 5-Fold Cross-Validation results. For a Decision Tree, `max_depth` can have a strong effect because it directly controls model complexity. For Random Forest, `n_estimators`, `max_depth`, and `max_features` can influence variance, diversity and complexity.

16.3 How does Random Forest improve generalization?

Random Forest combines multiple trees trained using bootstrap samples and random feature selection. Their predictions are combined through ensemble voting, reducing dependence on any single tree and generally reducing variance.

16.4 Did ensemble learning always improve performance? Why or why not?

No model family is guaranteed to outperform another on every dataset, split or metric. The conclusion should be based on the actual cross-validation and test-set results obtained in this experiment.

17 Result

The Decision Tree and Random Forest classifiers were implemented on the Wisconsin Diagnostic Breast Cancer dataset. Exploratory Data Analysis was performed using class distribution and feature correlation analysis. The dataset was divided into training and testing sets using an 80–20 split.

Five-Fold Cross-Validation was used for hyperparameter selection. The final models were evaluated using accuracy, precision, recall, F1-score, confusion matrix, ROC curve and AUC.

18 Conclusion

Decision Tree and Random Forest models were implemented and evaluated using 5-Fold Cross-Validation. Hyperparameters were selected based on average cross-validation performance, providing a robust approach to model selection.

The experiment demonstrates the difference between a single decision tree and an ensemble of decision trees. A Decision Tree is sensitive to model complexity and may overfit when grown excessively deep, while Random Forest reduces variance by combining multiple trees trained with bootstrap sampling and random feature selection.

19 References

1. Scikit-learn: Decision Trees: <https://scikit-learn.org/stable/modules/tree.html>
2. Scikit-learn: Random Forest: <https://scikit-learn.org/stable/modules/ensemble.html>
3. UCI Machine Learning Repository: <https://archive.ics.uci.edu>